

ALL PROGRAMMABLE

ANY MEDIA

5G

4K/8K

ANY STANDARD

ANY MACHINE

ANY NETWORK

5G Wireless • Embedded Vision • Industrial IoT • Cloud Computing



Introduction to Overlays



Agenda

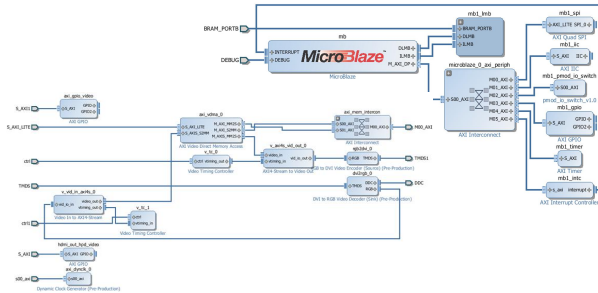
- Overlay Concept
- Base Overlay
- Labs:
 - Grove temperature sensor
 - Grove light sensor
 - Grove ledbar
 - OLED

Overlay Concept

FPGA overlays – hardware libraries (kernels)

- Overlays are generic FPGA designs that target multiple users with new design abstractions and tools
 - Post-bitstream programmable via software APIs
 - Typically optimized for given application domains
 - Encourages the use of open source tools & fast compilation
 - Enables productivity from re-using pre-optimized designs
 - Exposes benefits of FPGAs to new users

FPGA overlays – hardware libraries



Step 1:
Create an FPGA design for a class
of related applications

```

pmod_init(0,1);
while(1){
    while((MAILBOX_CMD_ADDR & 0x01) == 0){
        cmd=MAILBOX_CMD_ADDR;

        count = (cmd & 0x0000ff00) >> 8;
        if((count==0) || (count>253)) {
            // clear bit[0] to indicate
            // set rest to 1s to indicate
            MAILBOX_CMD_ADDR = 0xfffffff;
            return -1;
        }
        for(i=0; i<count; i++) {
            if (cmd & 0x08) // Python issue: read
            {
                switch ((cmd & 0x06) >> 1) { // use bit[2:1]
                    case 0 : MAILBOX_DATA(i) = *(u8 *) MAILBOX_ADDR; break;
                    case 1 : MAILBOX_DATA(i) = *(u16 *) MAILBOX_ADDR; break;
                    case 2 : break;
                    case 3 : MAILBOX_DATA(i) = *(u32 *) MAILBOX_ADDR; break;
                }
            }
        }
    }
}
    
```

6c78	3963	7367	3232	3500	6300	0b32
332f	3039	2f33	3000	6400	0931	323a
3a31	3900	6500	0532	7c	fff	ffff
ffff	ffff	ffff	ffff	ffa	9955	6630
0720	0031	a103	8031	413d	0831	6109
c204	0010	9330	e100	cf30	c100	8120
0020	0020	0020	0020	0020	0020	0020
0020	0020	0020	0020	0020	0020	0020
813c	c831	8108	8134	2100	0032	0100
e1ff	ff33	2100	0533	4100	0433	0101
6100	0032	8100	0032	a100	0032	c100

Step 2:
Export the bitstream and a C API
for programming the design

```

void setNormalDisplay(){
    sendCommand(OLED_Normal_Display_Cmd);
}

void setInverseDisplay(){
    sendCommand(OLED_Inverse_Display_Cmd);
}

int main(void)
{
    int cmd;
    int Row, Column;

    arduino_init(0,0,0,0);
    config_arduino_switch(A_GPIO, A_GPIO, A_GPIO,
        A_GPIO, A_SDA, A_SCL,
        D_GPIO, D_GPIO, D_GPIO, D_GPIO, D_GPIO,
        D_GPIO, D_GPIO, D_GPIO, D_GPIO,
        D_GPIO, D_GPIO, D_GPIO, D_GPIO);

    // Initialization
    oled_init();
}
    
```

Step 3:
Wrap the C API to create a Python
library

```

from time import sleep
from pyng import Overlay
from pyng.iop import PMOD_ADC, PMOD_DAC

ol = overlay("base.bit")
ol.download()
# Writing values from 0.0V to 2.0V with step 0.1V.
dac_id = int(input("Type in the PMOD ID of the DAC (1 ~ 2): "))
adc_id = int(input("Type in the PMOD ID of the ADC (1 ~ 2): "))

dac = PMOD_DAC(dac_id)
adc = PMOD_ADC(adc_id)

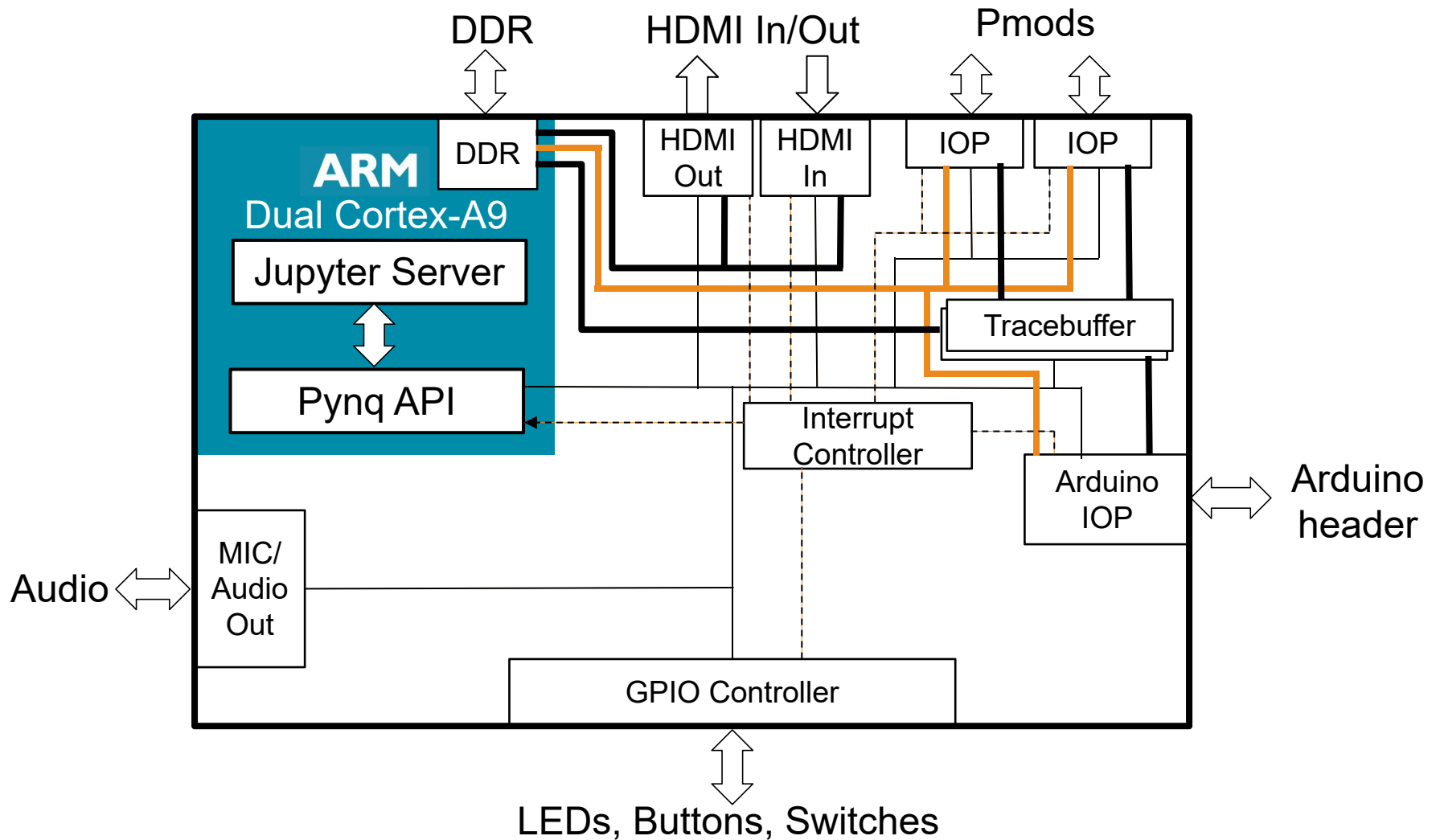
for j in range(20):
    value = 0.1 * j
    dac.write(value)
    sleep(0.5)
    # readings=adc.read(1,0,0)
    # x1=readings[0]
    print("Voltage read by DAC is: {:.4f} Volts".format(adc.read(1,0,0)[0]))
    
```

6c78	3963	7367	3232	3500	6300	0b32
332f	3039	2f33	3000	6400	0931	323a
f	fff	f	fff	f	fff	fff
5	6630					
1	6109					
0	8120					
0	0020					
0	0020					
2	0100					
3	0101					
2	c100					

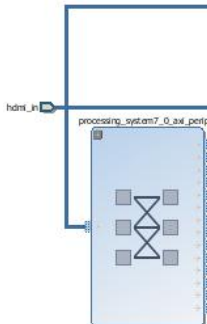
Step 4:
Import the bitstream and the library
in your Python scripts and program

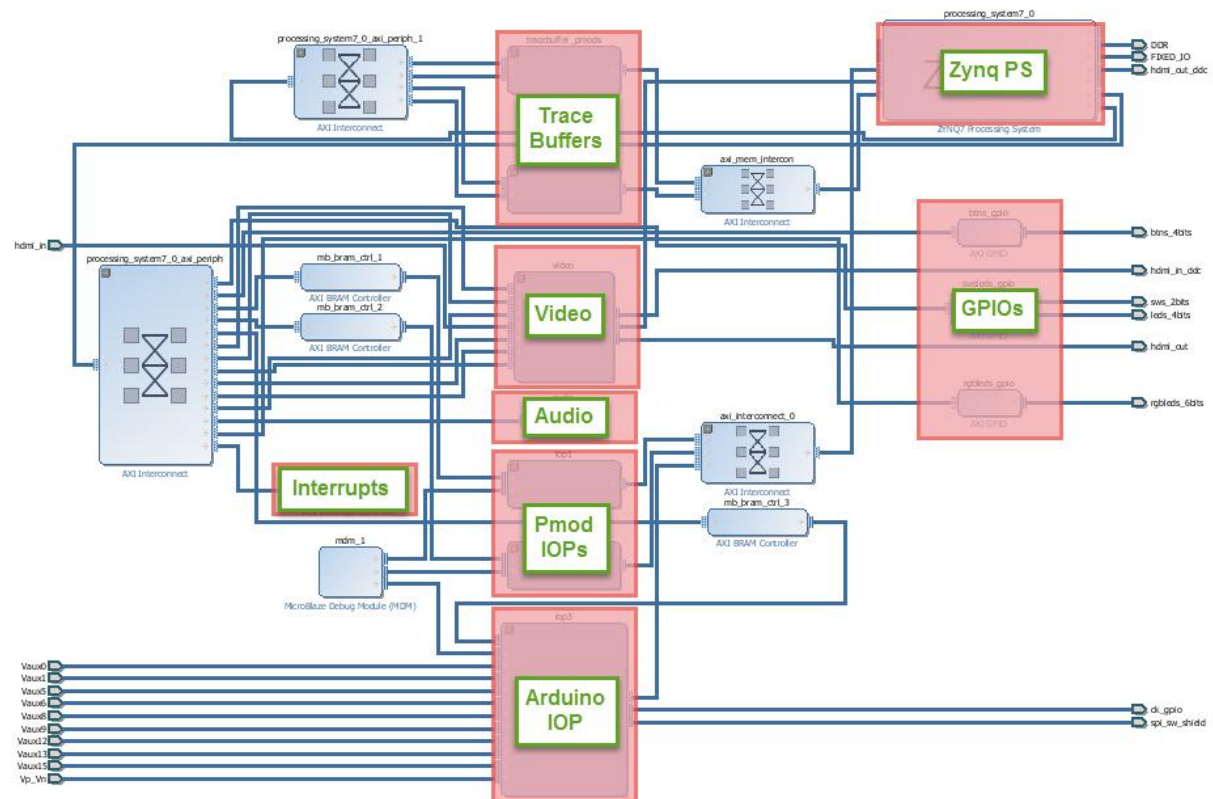
Base Overlay

Base Overlay (base.bit)



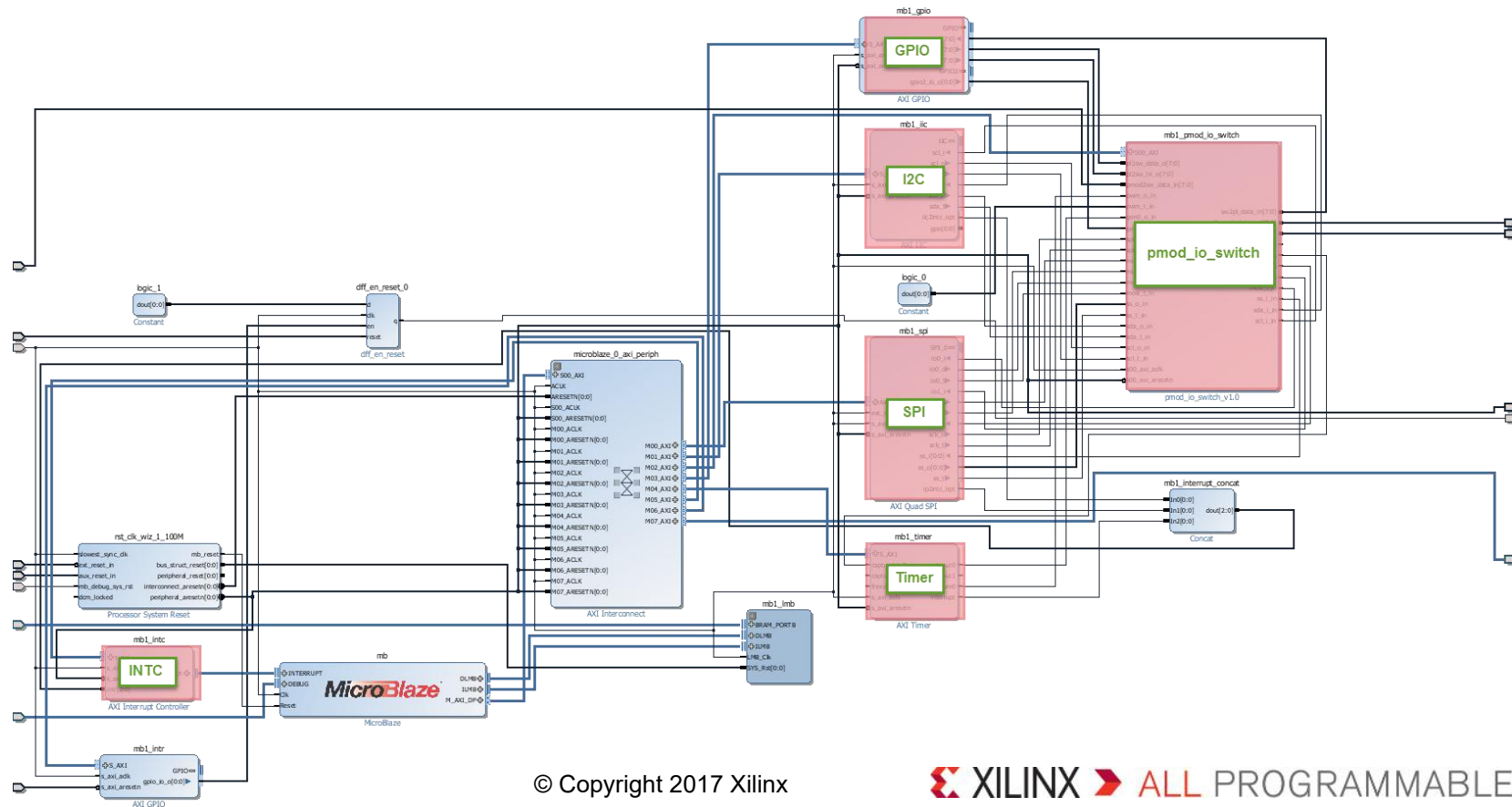
Base Overlay – Interface View

- **Base Overlay is PL design**
 - **Standard FPGA design flow**
 - Vivado IPI
 - **Interface to Python**
 - Memory Map
 - **Open source**
 - Components are re-useable
- 
- The diagram illustrates an FPGA architecture. A large blue rectangle represents the FPGA. Inside, there is a block labeled 'processing_system7_0_axi_periph'. To the left of this block, there is a port labeled 'hdmi_in'. A blue line connects the 'hdmi_in' port to the 'processing_system7_0_axi_periph' block. The block itself contains a schematic of a processing system with various components and interconnections.

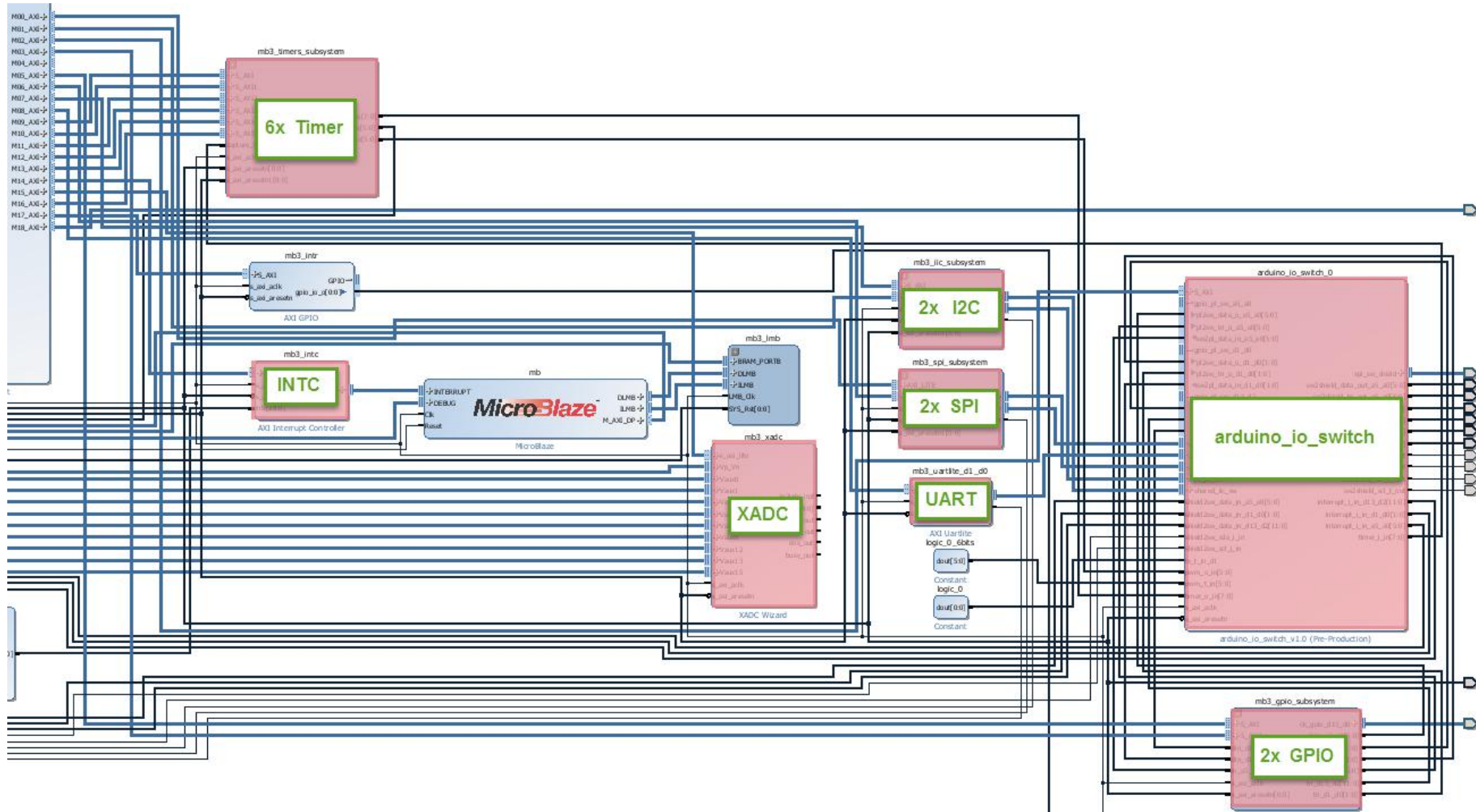


Pmod IOP

- Flexible block allows different peripherals to be used without new hardware design
- MicroBlaze processor sharing memory with PS
 - IIC, SPI, GPIO, Timer, Interrupt controller, Configurable Switch

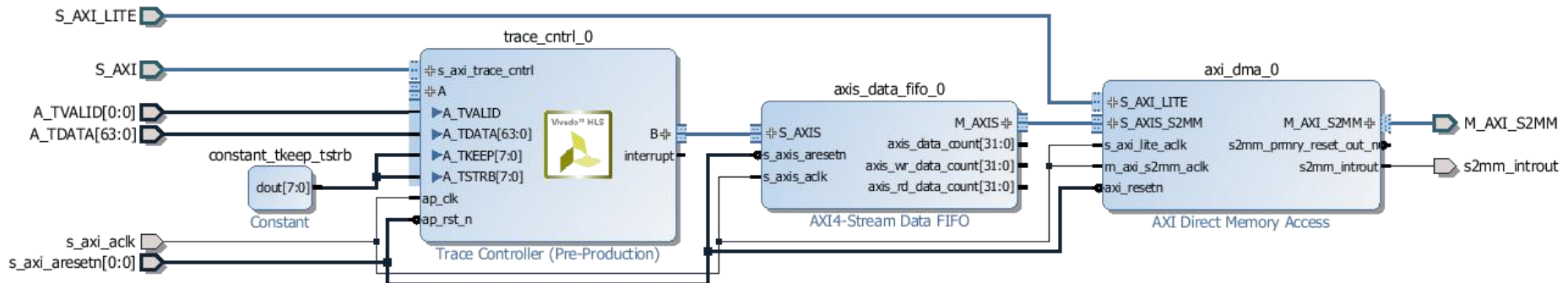


Arduino IOP



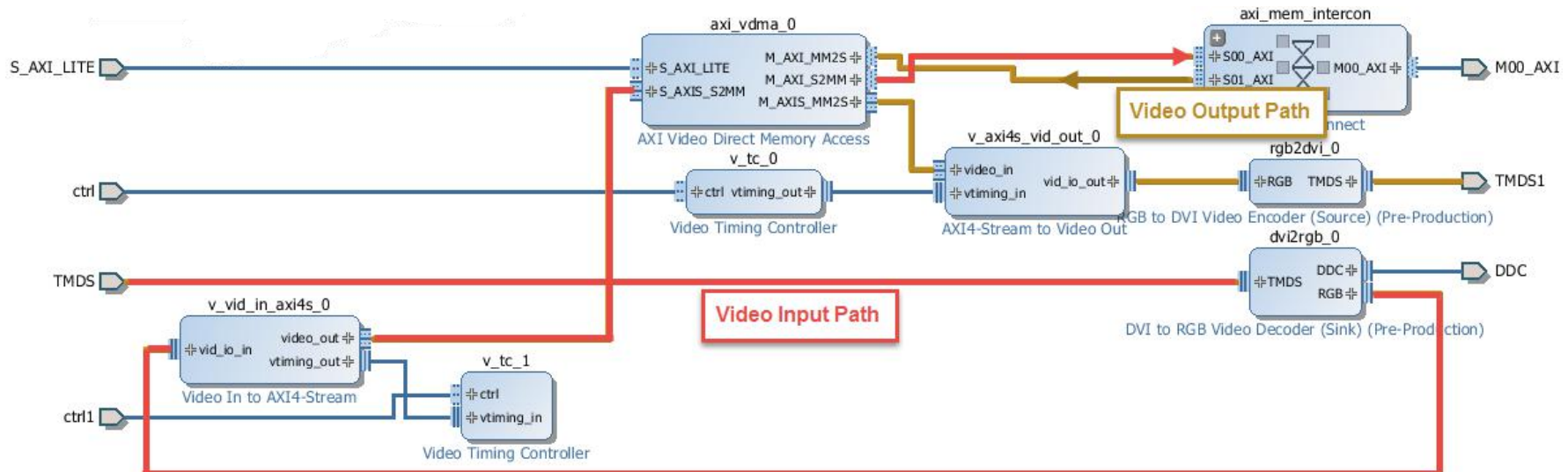
Tracebuffer IP

- Tracebuffer running at 100 MHz connected to IOP outputs
- Stream trace data into DDR memory
 - 4MB blocks via DMA
- 64-bit data acquisition
- Trigger Conditions
- Python control
 - Start/stop, Acquisition rate, Number of samples



Video

- HDMI In, HDMI Out
- HDMI input controller writes to a framebuffer
 - PS DDR memory
 - 3x framebuffers available
- Data path
 - Stream from HDMI in to DDR, Stream from DDR to HDMI out
- Image processing on PS
 - E.g. OpenCV

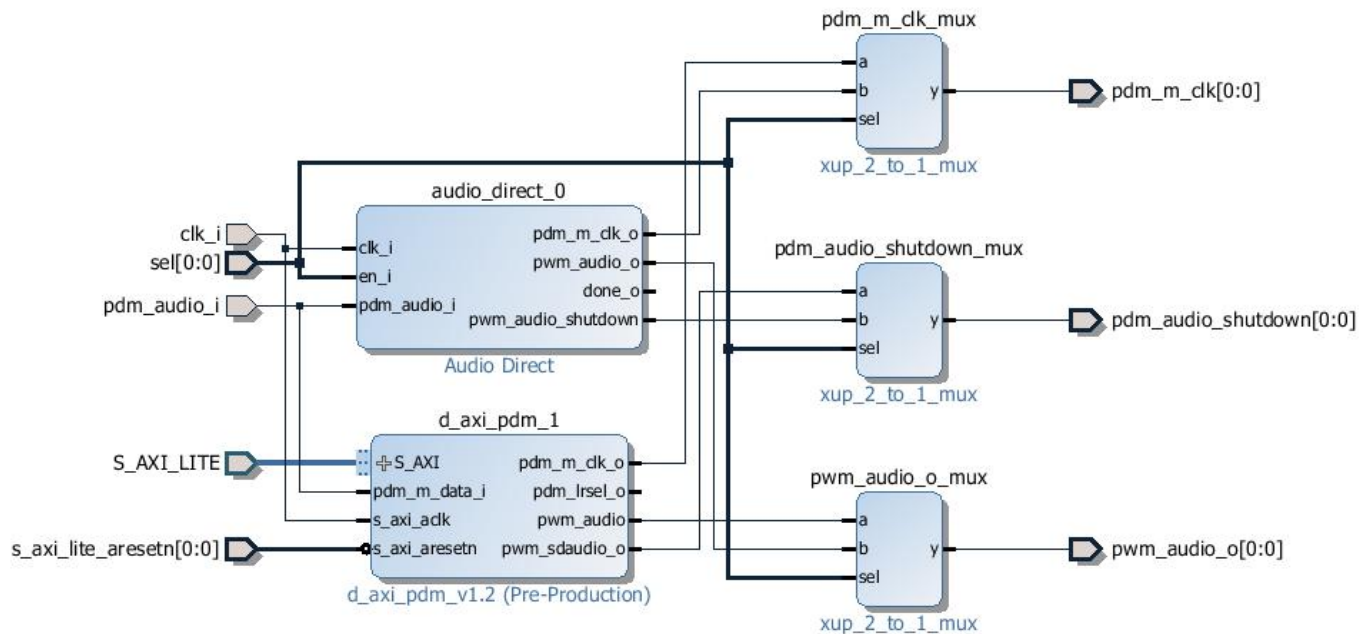


Audio

➤ Audio controller

- MIC in (PDM)
- Line Out (PWM)

➤ Connected to PS in Base Overlay



Base Overlay Resource Utilization

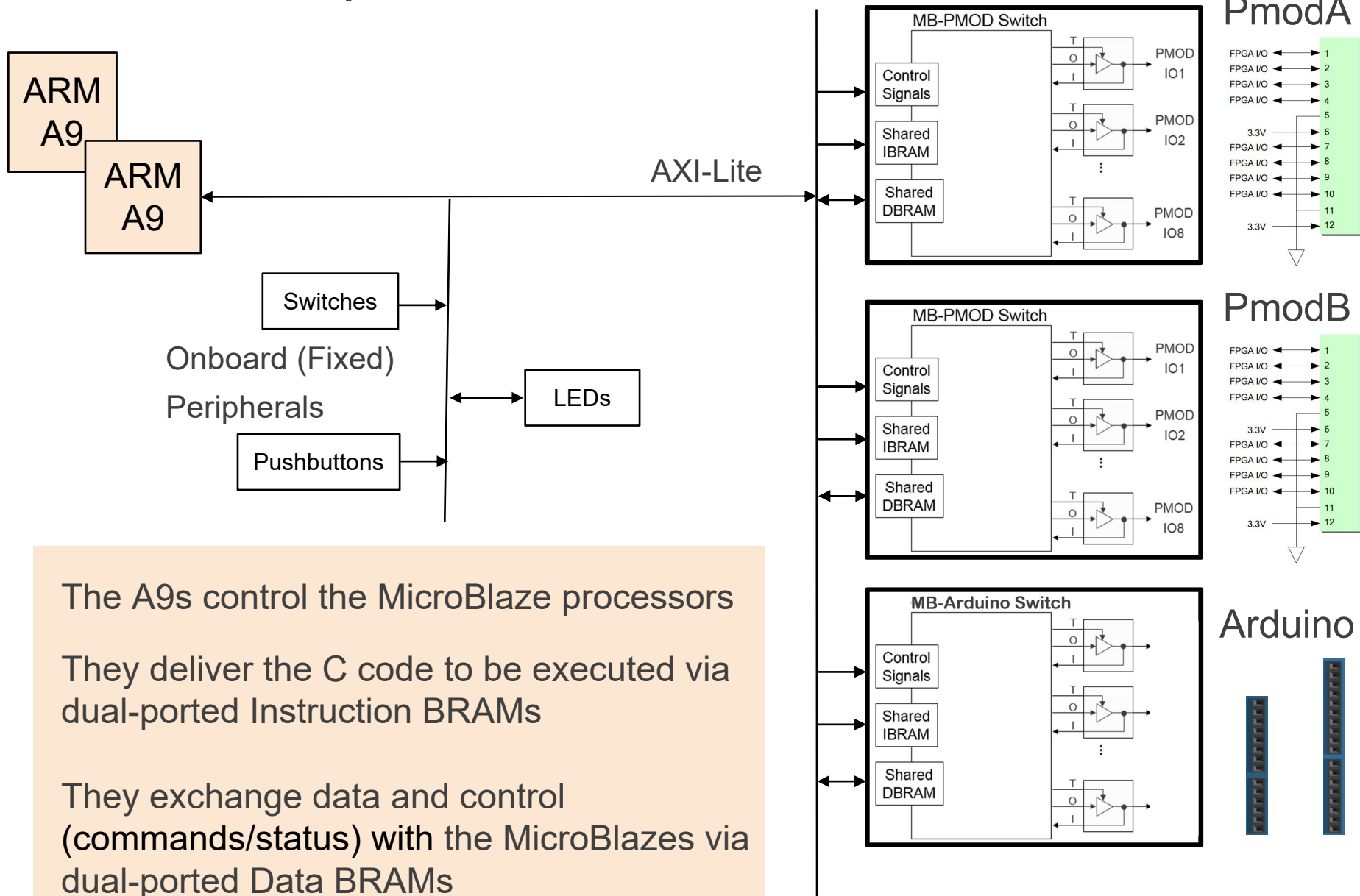
➤ Z-7020, Total resource utilization ~50%

- IOP (Pmod) ~ 5%
- Video ~ 10%
- Tracebuffer ~ 3%

IOP (Arduino) ~ 10%
Audio ~ 2%

Name	Slice LUTs (53200)	Slice Registers (106400)	F7 Muxes (26600)	F8 Muxes (13300)	Slice (13300)	LUT as Logic (53200)	LUT as Memory (17400)	LUT Flip Flop Pairs (53200)	Block RAM Tile (140)	DSPs (220)
top	30913	38295	826	102	11959	28400	2513	15573	68.5	6
system_i (system)	30913	38295	826	102	11959	28400	2513	15573	68.5	6
audio (audio_imp_P9M2FY)	1236	639	64	32	458	532	704	253	0	0
audio_path_sel (system_audio_pat...	0	0	0	0	0	0	0	0	0	0
axi_interconnect_0 (system_axi_int...	337	392	0	0	184	327	10	173	0	0
axi_mem_intercon (system_axi_m...	449	429	0	0	166	437	12	233	0	0
btns_gpio (system_btns_gpio_0)	75	99	0	0	36	75	0	60	0	0
concat_arduino (system_concat_ar...	0	0	0	0	0	0	0	0	0	0
concat_interrupts (system_concat_...	0	0	0	0	0	0	0	0	0	0
concat_pmods (system_concat_pm...	0	0	0	0	0	0	0	0	0	0
iop1 (iop1_imp_2RW50S)	2988	2749	123	2	1086	2839	149	1224	16	0
iop2 (iop2_imp_GME9RL)	2996	2749	123	2	1101	2847	149	1227	16	0
iop3 (iop3_imp_17QFVWL)	6146	5909	134	4	2137	5960	186	3002	16	0
iop_interrupts (system_iop_interru...	0	0	0	0	0	0	0	0	0	0
mb_1_intr_ack (system_mb_1_intr...	0	0	0	0	0	0	0	0	0	0
mb_1_reset (system_mb_1_reset_0)	0	0	0	0	0	0	0	0	0	0
mb_2_intr_ack (system_mb_2_intr...	0	0	0	0	0	0	0	0	0	0
mb_2_reset (system_mb_2_reset_0)	0	0	0	0	0	0	0	0	0	0
mb_3_intr_ack (system_mb_3_intr...	0	0	0	0	0	0	0	0	0	0
mb_3_reset (system_mb_3_reset_0)	0	0	0	0	0	0	0	0	0	0
mb_bram_ctrl_1 (system_mb_bra...	239	247	0	0	107	231	8	126	0	0
mb_bram_ctrl_2 (system_mb_bra...	239	247	0	0	114	231	8	127	0	0
mb_bram_ctrl_3 (system_mb_bra...	239	247	0	0	115	231	8	124	0	0
mdm_1 (system_mdm_1_0)	103	112	0	0	39	96	7	45	0	0
proc_sys_reset_142M (system_pro...	17	29	0	0	15	16	1	14	0	0
proc_sys_reset_pixeldk (system_p...	17	34	0	0	12	16	1	14	0	0
processing_system7_0 (system_pr...	2	0	0	0	2	2	0	0	0	0
processing_system7_0_axi_periph ...	5803	7740	124	62	2450	5061	742	2860	0	0
processing_system7_0_axi_periph...	587	695	0	0	241	520	67	286	0	0
rgbleds_gpio (system_rgbleds_gpi...	38	51	0	0	16	38	0	25	0	0
rst_processing_system7_0_100M (...)	17	29	0	0	11	16	1	15	0	0
rst_processing_system7_0_166M (...)	17	29	0	0	10	16	1	15	0	0
swslds_gpio (system_swslds_gp...	82	99	0	0	36	82	0	65	0	0
system_interrupts (system_system...	196	179	0	0	79	196	0	136	0	0
tracebuffer_arduino (tracebuffer_a...	1480	2417	3	0	644	1393	87	984	3	3
tracebuffer_pmods (tracebuffer_p...	1471	2387	3	0	626	1384	87	971	3	3
video (video_imp_1EQPUBJ)	6143	10787	252	0	2963	5858	285	3517	14.5	0

Base Overlay MicroBlaze IOP



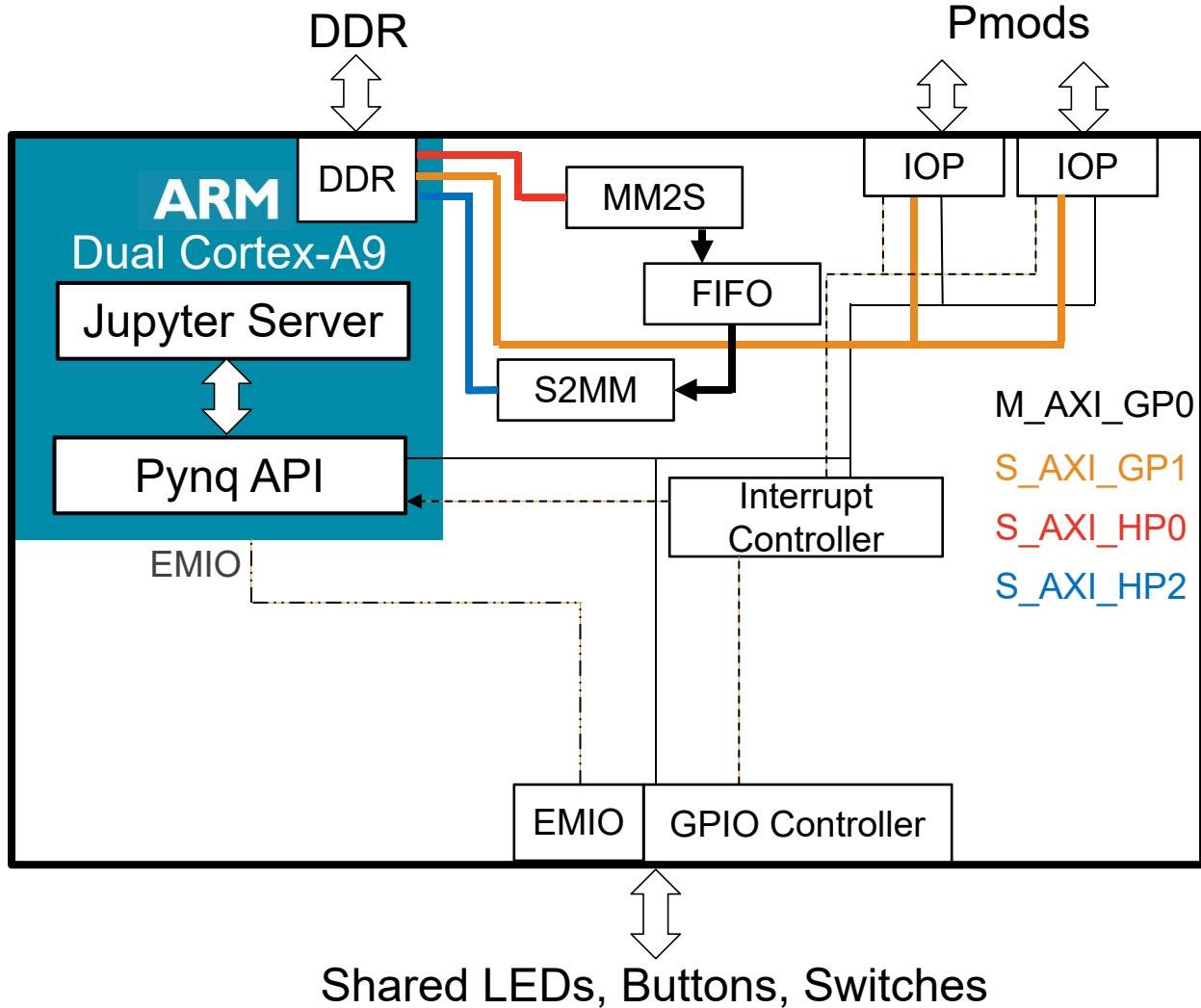
The A9s control the MicroBlaze processors

They deliver the C code to be executed via dual-ported Instruction BRAMs

They exchange data and control (commands/status) with the MicroBlazes via dual-ported Data BRAMs

Pynq_tutorial Overlay

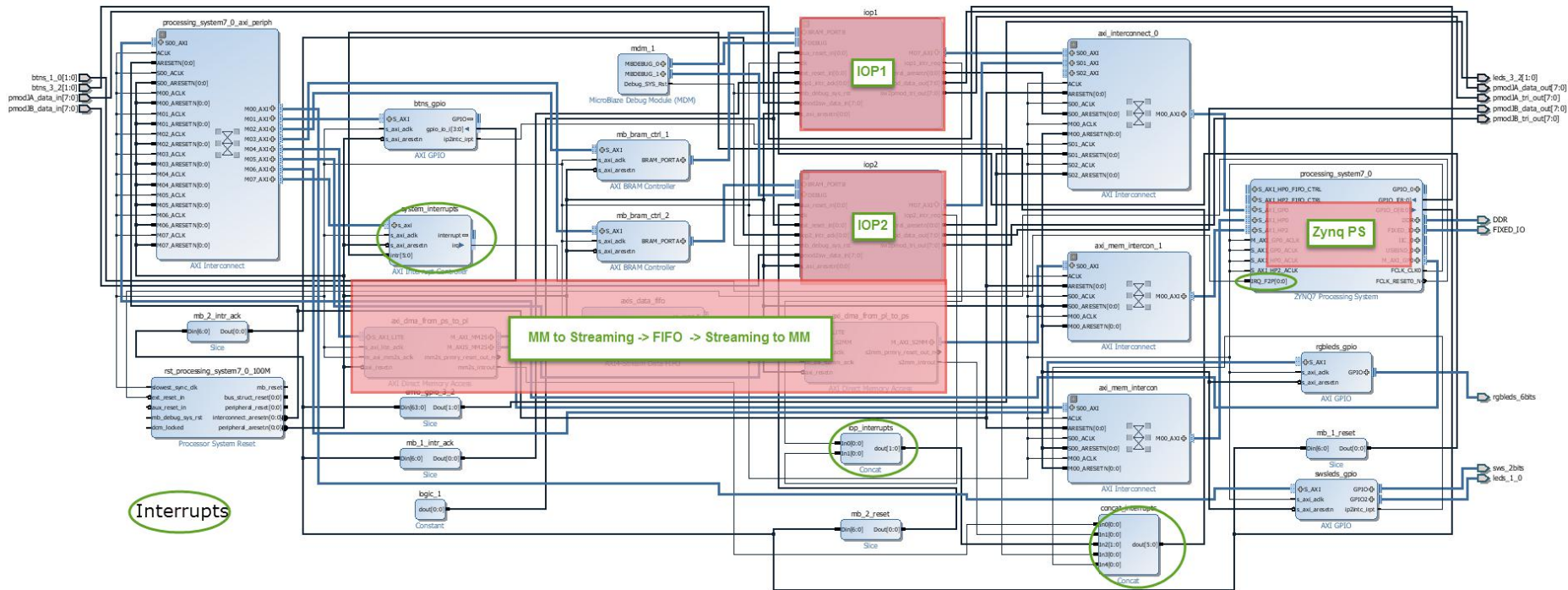
Pynq_tutorial Overlay (pynq_tutorial.bit)



Pynq_tutorial Overlay

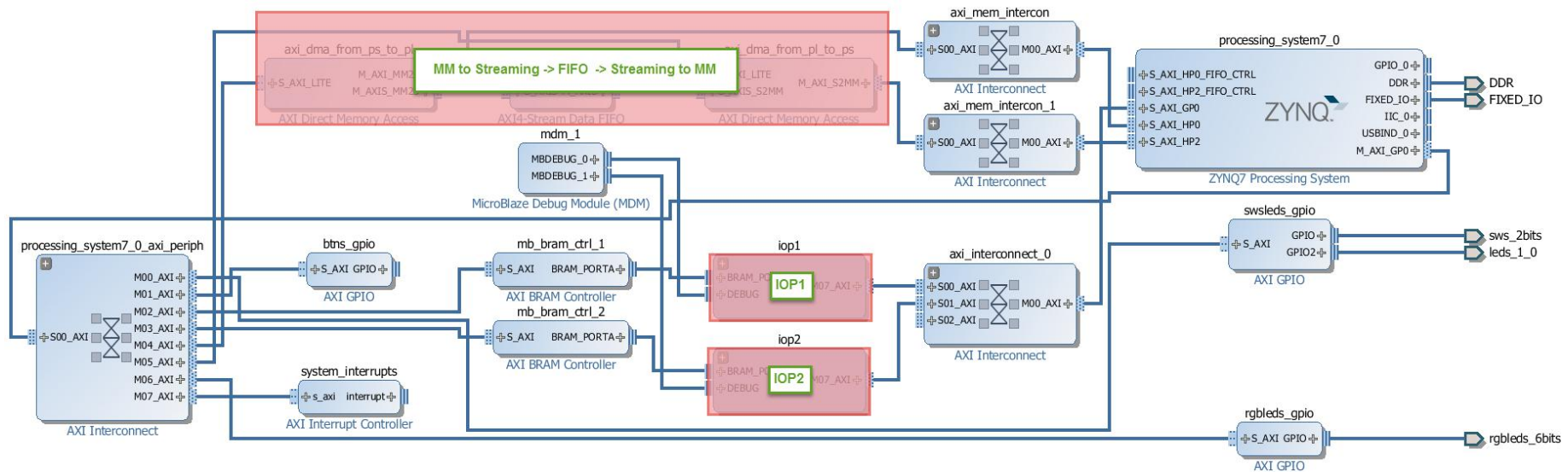
➤ Pynq_tutorial Overlay is a light-weight “base” overlay

- IOP1 and IOP2 included
- No Video, Audio, Arduino IOP, Tracebuffer
- MM to Streaming -> FIFO -> Streaming to MM added
- Shared LEDs and buttons between EMIO and MIO ports of PS



pynq_tutorial Overlay – Interface View

- Allows testing of DMA interfaces to AXI streams
 - Send and Receive DMAs
- Replace FIFO with custom stream IP
- Use pynq_tutorial overlay to test DMA operation



Pynq interface classes

➤ MMIO

- Memory Mapped Input Output
- Register based memory mapped transactions

➤ Xlnk

- Memory allocation (used in DMA)

➤ DMA (drivers)

- Direct Memory Access
- Offload memory transfers from main CPU

➤ GPIO

- Read/Write GPIO wires



Xilinx / PYNQ	
Code	Issues 0
Pull requests 0	Projects 0
Wiki	Pulse
Branch: master PYNQ / python / pynq /	
..	
_pynq	More specific error message for frame buffer allocation failure
board	Add Interrupt Support to PYNQ (#246)
drivers	fix issue when stop() attribute cannot be located
iop	Interrupt Refinements (#272)
tests	PL and IP driver updates (#243)
__init__.py	Add Interrupt Support to PYNQ (#246)
general_const.py	added bin files
gpio.py	PL and IP driver updates (#243)
interrupt.py	Add Interrupt Support to PYNQ (#246)
mmio.py	Improve performance of MMIO class (#229)
pl.py	Interrupt Refinements (#272)
xlnk.py	changing video and xlnk python API (#257)

MMIO

➤ Import MMIO

➤ Define memory mapped region

- BASE_ADDRESS: starting location
- ARRAY_SIZE: length of accessible memory

➤ Read and Write 32-bit values

- ADDRESS_OFFSET: Offset from BASE_ADDRESS
- Need to ensure the memory can be read/written

```
from pynq import MMIO

BASE_ADDRESS = 0x40000000
ARRAY_SIZE = 1024

mmio = MMIO(BASE_ADDRESS, ARRAY_SIZE)

data = 0x12345678
ADDRESS_OFFSET = 0x10

mmio.write(ADDRESS_OFFSET, data)

result = mmio.read(ADDRESS_OFFSET)

print(result)
> 0x12345678
```

Xlnk

- Memory managed by Linux
 - Virtual
- Memory must be allocated before it can be accessed by PL
 - PL needs the physical address of a memory buffer
- Xlnk can allocate (contiguous) memory buffers
 - Translates between virtual and physical
 - Contiguous memory is more efficient/allows simpler DMA logic
- Once buffer is allocated a DMA can be used to transfer data to/from PS/PL
- DMA class uses Xlnk for memory allocation

Xlnk example

- Import Xlnk
- Allocate contiguous buffer
 - cma_alloc()
 - Returns Linux virtual address
- Get Physical Address
 - cma_get_phy_addr()
 - Can be used by PL to access DDR (Linux) memory
- Write to buffer/Read from buffer

```
MEMORY_SIZE = 10
from pynq import Xlnk
mmu = Xlnk()

virt_ptr = mmu.cma_alloc(MEMORY_SIZE)
phy_addr = mmu.cma_get_phy_addr(virt_ptr)

buffer = mmu.cma_get_buffer(virt_ptr,
                             MEMORY_SIZE)

buffer = [0,1,2,3,4,5,6,7,8,9]

for i in range(10):
    print(buffer[i])
```

DMA

➤ Direct memory access

- Transfer data between memories directly
 - PS - PL
- Bypasses CPU
 - Doesn't waste CPU cycles on data transfer
- Speed up memory transfers with burst transactions

➤ Xilinx *AXI Direct Memory Access* IP block supported in PYNQ

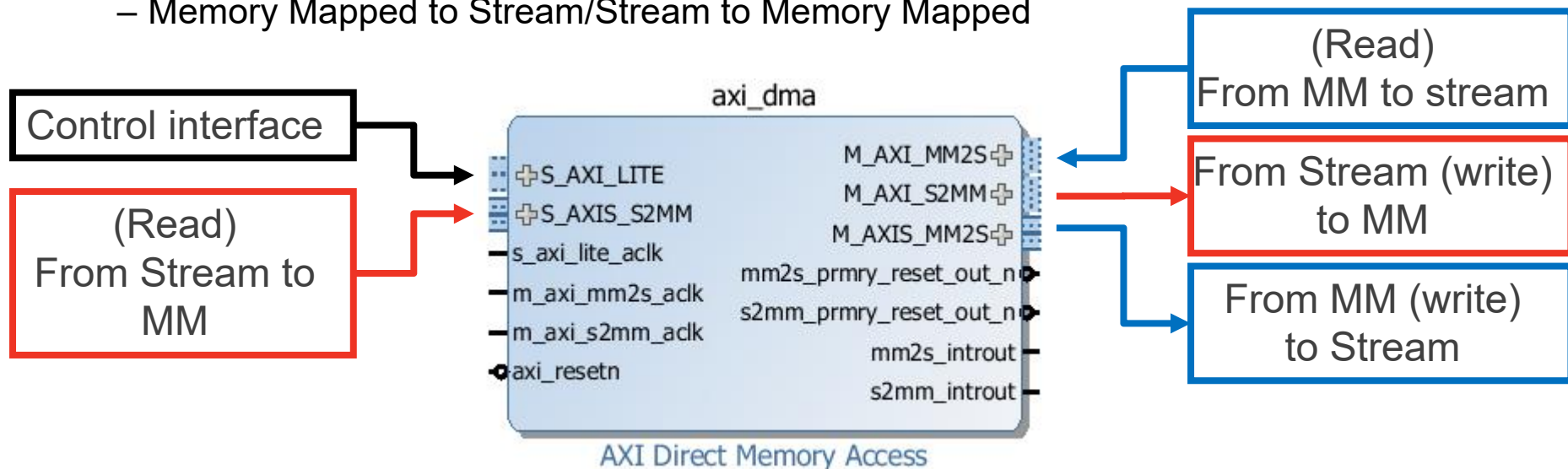
- Read and Write Paths from PL to DDR and DDR to PL
- Memory Mapped to Stream
- Stream to Memory Mapped

➤ Needs to stream to/from an allocated memory buffer

- PYNQ DMA class inherits from xlnk for memory allocation

AXI Direct Memory Access IP

- AXI Lite control interface (AXI GP port)
- Memory mapped interface (AXI interface, HP/ACP ports)
- AXI Stream interface (AXI stream accelerator)
- Transfer between streams and memory mapped locations
 - Paths from PL to DDR and DDR to PL
 - Memory Mapped to Stream/Stream to Memory Mapped



DMA example

- Setup DMAs
- Allocate memory buffers
- Start DMA transactions

```
from pynq.drivers import DMA
```

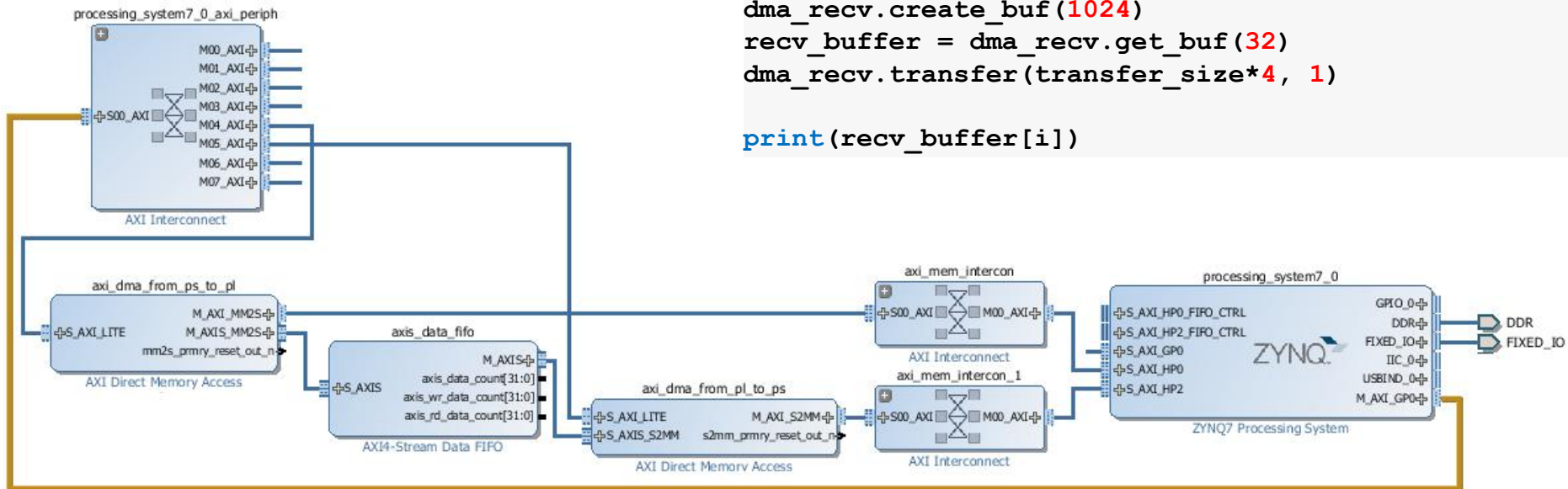
```
dma_send = DMA(dma_send_addr, 0) # DMA_TO_DEV
dma_recv = DMA(dma_recv_addr, 1) # DMA_FROM_DEV
```

```
dma_send.create_buf(1024)
send_buffer = dma_send.get_buf(32) # 32-bit
send_buffer[i] = ...
```

```
dma_send.transfer(transfer_size*4, 0) # Bytes
```

```
dma_recv.create_buf(1024)
recv_buffer = dma_recv.get_buf(32)
dma_recv.transfer(transfer_size*4, 1)
```

```
print(recv_buffer[i])
```



GPIO

- Up to 64 GPIO wires from PS
 - Tri-state
- Accessed from Linux
- Setup GPIO instance
- Map to GPIO pin
 - Translated pin numbers to Linux GPIO number
 - `get_gpio_pin()`
- Read/Write
- Most appropriate for simple control
 - Set, reset, start, done ...

```
from pynq import GPIO

btn_gpio = GPIO.get_gpio_pin(0)
led_gpio = GPIO.get_gpio_pin(1)

ps_btn = GPIO(btn_gpio, 'in')
ps_led = GPIO(led_gpio, 'out')

ps_btn.read()

ps_led.write()
```

Python Wrapper

- Standard Python code
- Interface classes used
 - MMIO, XInk, DMA, GPIO
- Check PYNQ repository for examples

```
class LED(object):
    """This class controls the onboard LEDs.

    Attributes
    -----
    index : int
        The index of the onboard LED, starting from 0.
    """
    _mmio = None
    _leds_value = 0

    def __init__(self, index):
        """Create a new LED object.

        Parameters
        -----
        index : int
            Index of the LED, from 0 to 3.

        """
        if not index in range(4):
            raise ValueError("Index for onboard LEDs should be 0 - 3.")

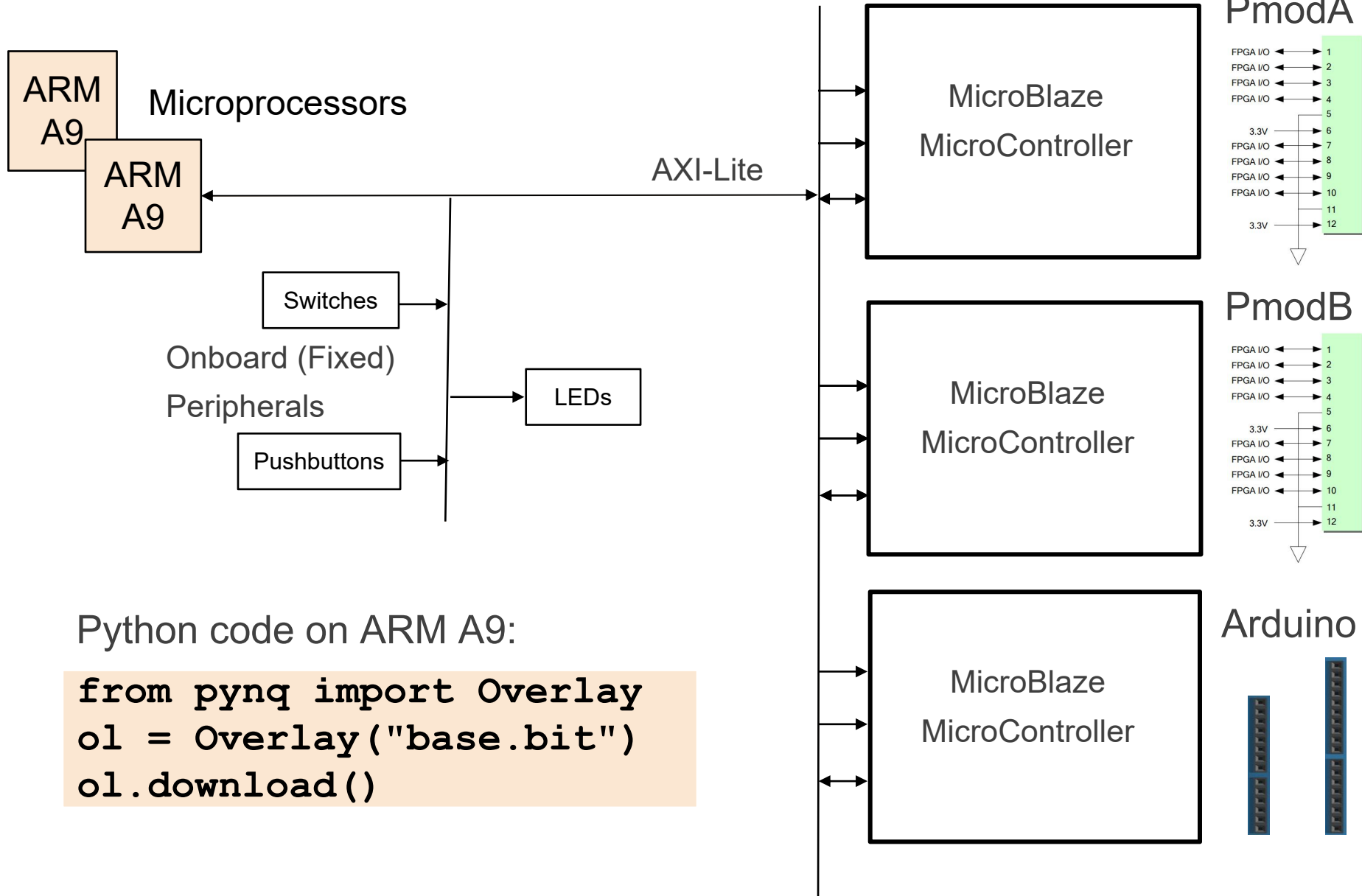
        self.index = index
        if LED._mmio is None:
            LED._mmio = MMIO(PL.ip_dict["SEG_swsleds_gpio_Reg"][0], 16)
        LED._mmio.write(LED_OFFSET1, 0x0)
```

Summary

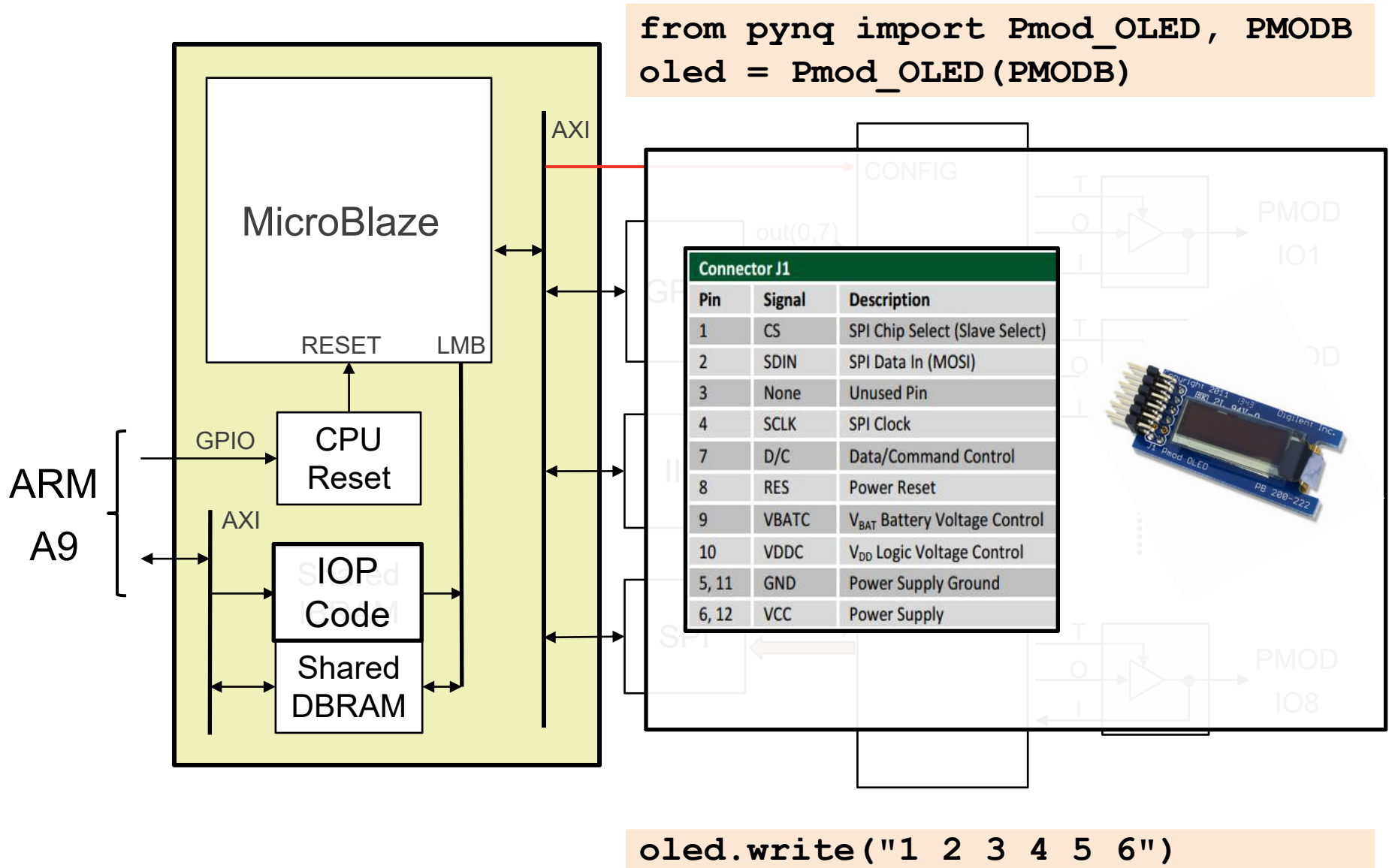
- Overlay Concept
- Base Overlay
- Pynq_tutorial Overlay
- Labs
 - GPIO example
 - Xlnk example
 - DMA example

Using Sensors

Load 'base' overlay on PL



Configure IO processor



Jupyter Notebook

```
In [1]: from pynq import PL, Overlay
        from pynq.iop import PMODB, Pmod_OLED

In [2]: ol = Overlay("base.bit")
        ol.download() # programs the Zynq FPGA

In [3]: oled = Pmod_OLED(PMODB)

In [4]: oled.write("1 2 3 4 5 6")
```

6 lines of user code ... thanks to Python, FPGA overlays,
abstraction & re-use

Summary

- Overlay Concept
- Base Overlay
 - Pynq_tutorial Overlay
- IOPs
- Using Sensors
- Labs
 - Grove temperature sensor
 - Grove light sensor
 - Grove LEDBar
 - PMOD OLED

Questions?